

UNIVERSIDADE DO VALE DO RIO DOS SINOS – UNISINOS

DIRETORIA DE EDUCAÇÃO CONTINUADA

ESPECIALIZAÇÃO EM ENGENHARIA DE SOFTWARE

Emanuel dos Santos Paz

Sistema de Notificação Inteligente para Detecção de Conflitos em Branches de Código

Porto Alegre

2025

UNIVERSIDADE DO VALE DO RIO DOS SINOS – UNISINOS

DIRETORIA DE EDUCAÇÃO CONTINUADA

ESPECIALIZAÇÃO EM ENGENHARIA DE SOFTWARE

Emanuel dos Santos Paz

Sistema de Notificação Inteligente para Detecção de Conflitos em Branches de Código

Trabalho apresentado à disciplina Engenharia de Software Aplicada, pelo Curso de Especialização em Engenharia de Software da Universidade do Vale do Rio dos Sinos - UNISINOS, ministrada pelo(a) professor(a) Josiane Brietzke Porto

Porto Alegre

2025

Sistema de Notificação Inteligente para Detecção de Conflitos em Branches de Código

Emanuel dos Santos Paz¹

¹ Instituto de Informática – Universidade do Vale do Rio dos Sinos (UNISINOS)

Av. Nilo Peçanha, 1600 – 91330-002 - Porto Alegre – RS – Brasil

PazEmanuel@edu.unisinos.br

Abstract. *This paper proposes the development of an intelligent notification system aimed at the early detection of code conflicts in branches, with the goal of reducing rework and improving communication among developers in distributed teams. The solution monitor commits made by other team members in Git repositories, identifying changes in files that are being simultaneously modified. The system alerts the user with relevant information, such as author, branch, and type of modification, even when changes occur in different branches. The research adopts an applied approach, focusing on the prevention of versioning failures and contributing to greater efficiency in collaborative software development processes.*

Resumo: *Este trabalho propõe o desenvolvimento de um sistema de notificação inteligente voltado à detecção antecipada de conflitos em branches de código, com o objetivo de reduzir retrabalho e melhorar a comunicação entre desenvolvedores em equipes distribuídas. A solução monitora os commits realizados por outros membros da equipe em repositórios Git, identificando alterações em arquivos que estejam sendo modificados simultaneamente. O sistema alerta o usuário com informações relevantes, como autor, branch e tipo de modificação, mesmo que as alterações ocorram em branches diferentes. A pesquisa adota uma abordagem aplicada, com foco na prevenção de falhas de versionamento, contribuindo para maior eficiência nos processos de desenvolvimento colaborativo.*

Palavras-chave: *Sistema de Notificação Inteligente, Detecção de Conflitos, Versionamento de Código, Desenvolvimento Colaborativo. Large Language Model (LLM)*

1. Introdução

Equipes de desenvolvimento distribuídas enfrentam dificuldades recorrentes com conflitos de código durante o processo de versionamento, especialmente em ambientes colaborativos que utilizam sistemas como Git, SVN e similares. A resolução desses conflitos pode degradar a qualidade do código mesclado e exige um profundo entendimento da estrutura e dos objetivos de um sistema (Ribeiro et al., 2022). Além disso, estudos indicam que 45,9% dos desenvolvedores enfrentam conflitos de merge em ambientes colaborativos, evidenciando a relevância do problema nas rotinas de desenvolvimento de software (Ribeiro et al., 2022). Por isso, a ausência de mecanismos proativos para detectar modificações simultâneas em arquivos, mesmo em branches diferentes, compromete a eficiência da comunicação entre desenvolvedores e aumenta o retrabalho. Esse cenário evidencia a necessidade de soluções que atuem preventivamente na identificação de conflitos, antes que eles se consolidem em merges problemáticos.

Nesse contexto, a presente pesquisa busca responder à seguinte questão: como um sistema de notificação inteligente pode auxiliar na detecção antecipada de conflitos em branches de código, reduzindo retrabalho e melhorando a comunicação entre desenvolvedores?

O objetivo geral deste trabalho é desenvolver um sistema de notificação inteligente capaz de monitorar os commits remotos realizados por outros membros da equipe, com o intuito de identificar alterações em arquivos que estejam sendo modificados pelo usuário localmente. A proposta visa antecipar possíveis conflitos de código, mesmo em branches diferentes, promovendo maior eficiência na comunicação entre desenvolvedores e na prevenção de falhas de versionamento. Os objetivos específicos incluem:

- Monitorar os commits remotos realizados por diferentes membros da equipe, com foco em arquivos que estejam sendo modificados localmente pelo usuário.
- Detectar alterações simultâneas em arquivos compartilhados, mesmo em branches distintos, e notificar o usuário com informações relevantes, como autor, branch, tipo de modificação e descrição do commit.
- A partir dos dados coletados pelo sistema, analisar os arquivos alterados e verificar se existe um possível conflito, dando justificativas e informando para o usuário quais são as alternativas plausíveis para a solução do conflito

A escolha por abordar esse problema surgiu a partir da observação de conflitos recorrentes em ambientes de versionamento SVN, vivenciados na empresa onde o autor atua. Esses conflitos, muitas vezes causados por modificações simultâneas em arquivos compartilhados, resultavam em retrabalho, perda de produtividade e falhas de comunicação entre os desenvolvedores.

Para investigar e validar a proposta, esta pesquisa adotou uma abordagem qualitativa, de natureza aplicada e com caráter exploratório. O método utilizado foi o estudo de caso tecnológico, fundamentado em uma situação real vivenciada na empresa onde o autor atua, envolvendo conflitos recorrentes em ambientes de versionamento SVN. A coleta e análise de dados foi realizada por meio de uma revisão gráfica gerada pela ferramenta de gerenciamento do repositório da empresa, complementada por observações contínuas sobre a maneira como os desenvolvedores lidam com conflitos de merge.

Este artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica; a Seção 3 descreve a metodologia adotada para validar a solução proposta; a Seção 4 detalha o desenvolvimento do sistema, incluindo as tecnologias utilizadas, a arquitetura definida e os requisitos funcionais atendidos; a Seção 5 apresenta os resultados obtidos; a Seção 6 discute esses resultados; a Seção 7 apresenta a conclusão do projeto; e a Seção 8 contém os agradecimentos.

2. Fundamentação Teórica

2.1. Controle de Versão e Git

O controle de versão é uma prática essencial no desenvolvimento de software moderno, permitindo que múltiplos desenvolvedores trabalhem simultaneamente em diferentes partes de um projeto sem comprometer a integridade do código. Sistemas como o Git se destacam por sua abordagem distribuída, na qual cada colaborador possui uma cópia completa do repositório, possibilitando maior flexibilidade e autonomia no processo de desenvolvimento. O Git organiza o histórico de alterações por meio de commits e permite a criação de branches para isolar funcionalidades ou correções, promovendo paralelismo e organização no fluxo de trabalho (Chacon & Straub, 2014).

2.2. Conflitos de Código em Ambientes Colaborativo

Apesar das vantagens do Git e outros versionamentos, o uso intensivo de branches pode gerar conflitos de código, os fatores que mais levam a conflitos são o tempo de isolamento da branch e a falta de comunicação (Ribeiro et al., 2022). Podendo resultar em retrabalho, atrasos e falhas na integração contínua. Segundo Brun et al. (2011), de tal forma como acontece com os erros nos programas, geralmente é mais fácil e barato identificar e corrigir conflitos precocemente, antes que eles se propaguem no código e as alterações relevantes desapareçam da memória dos desenvolvedores.

2.3. Sistemas de Notificação Inteligente

Sistemas de notificação inteligente têm sido utilizados em diversas áreas da computação para alertar usuários sobre eventos relevantes, com base em critérios definidos ou aprendizado de padrões. No contexto do desenvolvimento de software, essas notificações podem ser aplicadas para informar sobre alterações simultâneas em arquivos, potenciais conflitos ou atividades em branches correlacionadas. Segundo Adepoju et al. (2022), o avanço dos sistemas de monitoramento e alerta em ecossistemas de dados complexos reflete uma transformação significativa na forma como as organizações abordam a confiabilidade dos sistemas e a prevenção de incidentes. A implantação de monitoramento contínuo e notificações pró-ativas tem promovido melhorias expressivas na experiência operacional, na colaboração entre desenvolvedores e no desempenho geral dos sistemas.

2.4. Trabalhos Relacionados

O artigo Understanding and Analyzing Factors that Affect Merge Conflicts from the Perspective of Brazilian Software Developers (Ribeiro et al., 2022) relata uma investigação empírica sobre os fatores que levam ou ajudam a evitar conflitos de *merge* no desenvolvimento colaborativo de *software*. O estudo baseou-se em uma pesquisa de levantamento (*survey*) com 109 desenvolvedores e entrevistas em campo com 15 desenvolvedores brasileiros.

O estudo conclui que o uso de branches é amplamente adotado para criar novas funcionalidades e corrigir bugs. Conflitos de merge acontecem com frequência moderada, mas geralmente são resolvidos sem grande dificuldade e em menos de 24 horas. Os principais fatores associados à ocorrência de conflitos são o longo tempo em que uma branch permanece isolada e a falta de comunicação entre os desenvolvedores, que muitas vezes acabam trabalhando na mesma funcionalidade sem perceber.

O artigo Advancing Monitoring and Alert Systems: A Proactive Approach to Improving Reliability in Complex Data Ecosystems (Adepoju et al., 2022) aborda um estudo de modernizar os sistemas de monitoramento e alerta em ecossistemas de dados que se tornaram complexos devido a infraestruturas baseadas em nuvem, IoT e *big data*. O artigo defende a transição de abordagens reativas (que detectam problemas após ocorrerem) para abordagens proativas.

O estudo propõe e justifica um framework avançado que inclui:

- Monitoramento em Tempo Real: Essencial para detecção instantânea de anomalias, minimizando a latência e o tempo de resposta em ambientes dinâmicos e de alta velocidade de dados

- **Análise Preditiva e Machine Learning:** Utiliza dados históricos, modelos de *machine learning* e algoritmos de reconhecimento de padrões para prever potenciais falhas ou interrupções antes que ocorram, permitindo intervenções oportunas e a geração de alertas precoces
- **Estratégias de Prevenção de Incidentes:** Reforçadas pela automação inteligente e limiares adaptativos, que ajustam-se dinamicamente com base em dados em tempo real, mantendo o desempenho ideal do sistema e reduzindo *downtime*
- **Colaboração e Integração:** O modelo holístico proposto promove a colaboração entre stakeholders, garantindo o alinhamento do sistema com os objetivos organizacionais e operacionais. A implementação dessas metodologias avançadas demonstrou melhorias tangíveis na confiabilidade operacional em diversos setores.

Os dois estudos analisados foram fundamentais para embasar e validar a proposta deste projeto aplicado. O trabalho de Ribeiro et al. (2022) reforça a premissa central de que conflitos de merge são fortemente influenciados por fatores humanos e organizacionais — especialmente o prolongado isolamento de branches e a falta de comunicação entre desenvolvedores. Esses achados sustentam a motivação do projeto aplicado, cuja finalidade é antecipar conflitos antes que eles se manifestem no processo de integração. Complementarmente, o estudo de Adepoju et al. (2022) valida o princípio da atuação proativa em sistemas complexos, fornecendo suporte teórico para o uso de mecanismos inteligentes — neste caso, LLMs — como estratégia para identificar divergências e auxiliar na resolução de conflitos. Dessa forma, ambos os artigos convergem para os objetivos específicos deste trabalho e justificam a relevância da solução proposta.

3. Metodologia

3.1. Delineamento de Pesquisa

A presente pesquisa adota uma abordagem qualitativa, de natureza aplicada e com caráter exploratório. O método utilizado é o estudo de caso tecnológico, fundamentado em uma situação real ocorrida em ambiente corporativo, onde conflitos em branches de código geraram retrabalho e atrasos no processo. A pesquisa tem como foco o desenvolvimento e validação funcional de um sistema inteligente de notificação, capaz de antecipar modificações simultâneas em arquivos versionados. A escolha da abordagem qualitativa se justifica pela necessidade de compreender o comportamento do sistema frente a cenários reais de conflito, interpretando os resultados com base na observação direta e na análise funcional da ferramenta.

3.2. Unidade de Análise

As unidades de análise desta pesquisa são os conflitos de código em ambientes de versionamento distribuído e o comportamento dos desenvolvedores diante desses problemas. O estudo foi fundamentado em um caso real ocorrido na empresa onde o autor atua, no qual a propagação excessiva de branches e a ausência de mecanismos claros de comunicação e controle culminaram em dificuldades significativas durante o processo de integração. Esse cenário resultou em atrasos na entrega de funcionalidades, retrabalho e elevado nível de estresse entre os desenvolvedores responsáveis pela fusão das branches na principal. Observou-se também uma sobrecarga operacional desses profissionais, evidenciando a necessidade de uma solução proativa para antecipação de conflitos e melhoria na coordenação entre membros da equipe.

3.3. Coleta e Análise de Dados

3.3.1. Coleta de Dados

A coleta de dados foi conduzida por meio de uma revisão gráfica realizada com a ferramenta de versionamento **SVN**, aplicada ao repositório referente ao estudo de caso do autor. Esse procedimento possibilitou uma análise aprofundada das dificuldades gerais observadas, permitindo maior rigor na interpretação dos resultados.

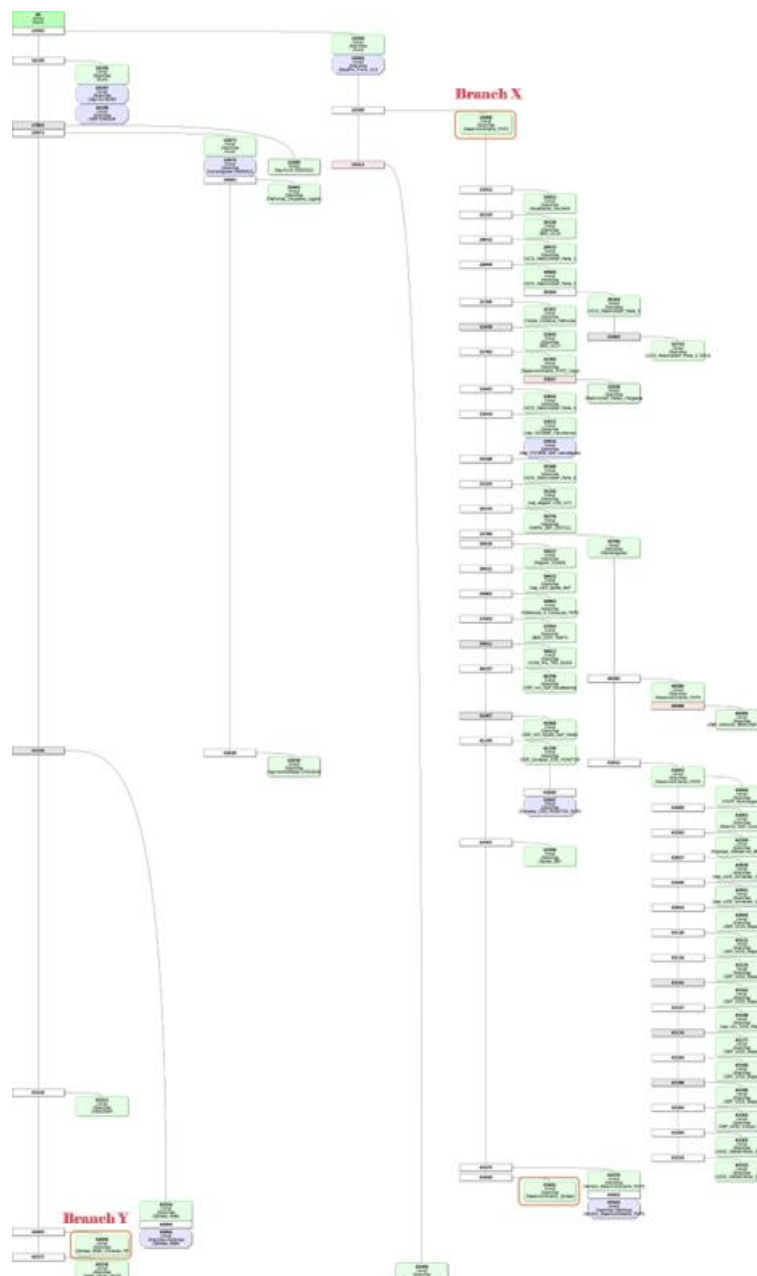


Figura 1. Gráfico do Repositório do caso de uso

A imagem ilustra dois ramos de desenvolvimento: à direita, encontra-se a *branch* mantida pela empresa terceirizada X, composta por diversas *branches* filhas; à esquerda, observa-se a *branch* única desenvolvida pela empresa terceirizada Y.

A pesquisa concentra-se na análise das *branches* correspondentes às revisões nº 44096 (ramo principal da empresa X, desenvolvido em 2023 e encerrado em 2024 devido à quebra de contrato) e nº 16600 (ramo principal da empresa Y, iniciado em 2025 e em desenvolvimento até o presente momento).

É importante destacar que a arquitetura de implantações deste projeto adota um modelo baseado em pacotes. Assim, cada implementação não é inserida diretamente na *branch trunk*. Em vez disso, realiza-se uma Sprint específica, cujo objetivo é selecionar quais *branches* devem ser integradas à *branch* de produção.

Em setembro de 2025, foi decidido que uma Sprint seria dedicada para dois desenvolvedores fazerem merge de algumas features da branch Y para branch de produção junto com a implementações da branch X. Dessa forma foi observado os seguintes comportamentos:

- Os desenvolvedores levaram três semanas para o entendimento do problema e uma semana para a solução com evidências concretas. Apresentando um atraso de uma Sprint (sabendo que uma Sprint possui duas semanas de desenvolvimento)
- Foi observado o estresse dos desenvolvedores ao longo da tarefa
- Foi observado a dificuldade dos desenvolvedores sobre o uso da ferramenta SVN para o merge das tarefas
- Registrou-se ausência dos desenvolvedores em momentos de tentativa de comunicação.

3.3.2. Análise de Dados

Embora o paradigma baseado em pacotes proporcione maior controle sobre o processo de integração, ele também apresenta desafios relevantes, como o aumento da probabilidade de conflitos entre branches, maior complexidade na coordenação da equipe e a necessidade de tempo adicional para implantação. Esses fatores podem gerar atrasos nas entregas, reduzir a produtividade e aumentar o retrabalho. Segundo Ribeiro et al. (2022), o tempo prolongado de isolamento das branches e a insuficiência de comunicação entre os desenvolvedores são elementos determinantes para o surgimento de conflitos de merge, evidenciando um dos principais pontos negativos desse paradigma

Observou-se que a falta de clareza em relação ao ecossistema do repositório durante o desenvolvimento do projeto contribuiu para a formação de um **débito técnico**, refletido diretamente nos dados coletados. Além disso, a ausência de uma monitoração adequada das demais entregas resultou em um **gargalo de trabalho**, que ocasionou estresse na equipe, acúmulo de tarefas e, por consequência, atrasos na conclusão das homologações.

Dado Coletado	Interpretação	Impacto	Risco
---------------	---------------	---------	-------

Dado Coletado	Interpretação	Impacto	Risco
Atraso de uma Sprint (3 semanas para entendimento + 1 semana para solução)	O tempo de resolução foi superior ao ciclo previsto, indicando baixa eficiência no processo de integração.	Comprometimento do cronograma e atraso nas entregas.	Acúmulo de tarefas futuras e aumento do débito técnico.
Estresse dos desenvolvedores	Sobrecarga e pressão decorrentes da complexidade do merge e da falta de clareza no repositório.	Redução da motivação e da qualidade do trabalho.	Possível rotatividade da equipe e queda de produtividade.
Dificuldade no uso da ferramenta SVN	Limitações técnicas ou falta de capacitação para lidar com merges complexos.	Maior tempo gasto em tarefas operacionais e risco de erros.	Retrabalho e falhas na integração de funcionalidades.
Ausência em momentos de comunicação	Falhas de alinhamento e coordenação entre os envolvidos.	Dificuldade no acompanhamento das atividades e perda de sincronia.	Desalinhamento organizacional e aumento de conflitos internos.

4. Desenvolvimento

O ponto de partida de desenvolvimento está na identificação e resolução de problemas relacionados à análise de dados, considerando uma arquitetura de implantações baseada em pacotes. Busca-se apresentar, de maneira clara, os possíveis conflitos de *merge* entre arquivos modificados localmente e aqueles presentes nas *branches* ainda não homologadas quando comparados à *branch* principal. Com base nesse objetivo, as seguintes etapas foram definidas para o desenvolvimento do projeto aplicado:

1. Definição dos Requisitos Funcionais da aplicação
2. Seleção das tecnologias e ferramentas que serão utilizadas na implementação

3. Definição da arquitetura do sistema, contemplando camadas, módulos e a lógica de processamento dos dados.
4. Implementação do Notificador, contemplando a aplicação das tecnologias selecionadas, a materialização da arquitetura definida e o atendimento aos requisitos funcionais especificados.
5. Criação de um repositório de testes, destinado a validar o funcionamento da aplicação em diferentes cenários de conflito.

4.1. Requisitos Funcionais

Diante dos dados coletados e da metodologia previamente definida, foi desenvolvido um módulo que atua como estrutura base para o Sistema de Notificação Git. O objetivo desse módulo é monitorar os commits remotos realizados pelos diversos membros da equipe, com ênfase na detecção de alterações em arquivos que estão sendo modificados localmente pelo usuário. A seguir, apresentam-se os principais requisitos funcionais que o sistema deve atender.

RF01 - O sistema deve solicitar o acesso do repositório,

RF02 - O sistema deve solicitar qual é a principal branch de desenvolvimento

RF03 - O sistema deve solicitar qual é o alias principal do repositório remoto (sendo origin o padrão).

RF04 - O sistema deve solicitar o Token de autenticação necessário para o uso da API do Gemini

RF05 - O sistema deve encapsular os possíveis conflitos de *merge* em um objeto estruturado, contendo as seguintes informações:

- a) Branch remota envolvida no processo de *merge*
- b) Lista de commits que apresentam potenciais conflitos
- c) Lista de arquivos com conflitos, cada um contendo:
 - a. *Diff* da branch principal (estado de referência)
 - b. *Diff* local (alterações realizadas na branch atual)
 - c. *Diff* remoto (alterações provenientes da branch remota)

- d. Informação textual indicando se há conflito, acompanhada de justificativa e possíveis soluções para o problema identificado

RF06 - O sistema deve gerar monitoramento abrangente de todos os processos da arquitetura

4.2. Tecnologias Utilizadas

A escolha da linguagem C# para o desenvolvimento do sistema considerou critérios como estabilidade, maturidade tecnológica e aderência ao contexto acadêmico e corporativo. Soma-se a isso a familiaridade prévia do autor com a linguagem, o que contribuiu para um processo de desenvolvimento mais ágil, organizado e alinhado ao escopo desta pesquisa. A linguagem oferece tipagem segura, gerenciamento automático de memória e recursos modernos — como *Language Integrated Query* (LINQ) e *async/await* — que aumentam a produtividade e favorecem a escrita de código claro e eficiente. Além disso, sua forte integração com o ecossistema .NET e a ampla disponibilidade de bibliotecas e ferramentas consolidam sua relevância na indústria, sendo continuamente aprimorada pela Microsoft e sustentada por uma comunidade ativa (MICROSOFT, 2025; STACK OVERFLOW, 2024).

Para a manipulação dos dados provenientes do repositório Git, utilizou-se a biblioteca LibGit2Sharp, amplamente adotada em aplicações .NET por oferecer uma interface robusta, segura e de alto nível para interação com repositórios. A escolha dessa biblioteca não se restringe à execução de comandos Git; ela funciona como uma camada de abstração que viabiliza a arquitetura proposta, possibilitando o acesso estruturado a *branches*, *commits*, *diffs* e demais metadados relevantes ao processo de detecção de conflitos. Sua maturidade e constante evolução pela comunidade reforçam sua adequação tanto para projetos acadêmicos quanto corporativos (LIBGIT2SHARP, 2025).

A biblioteca também permite encapsular os resultados em objetos bem definidos, possibilitando representar de forma sistemática as informações relacionadas aos conflitos de merge. Esse encapsulamento contribui para a clareza analítica do sistema e se alinha às boas práticas de engenharia de software, ao separar a lógica de manipulação dos dados da lógica de apresentação e resolução.

Por fim, para a comparação entre diferentes versões de arquivos, utilizou-se a API Gemini, que fornece mecanismos de análise de *diffs* por meio de modelos de linguagem. Essa abordagem possibilita identificar divergências entre o arquivo remoto e o arquivo local de maneira automatizada e estruturada (GOOGLE, 2024). Além da detecção das diferenças, a API também oferece explicações detalhadas sobre as modificações encontradas e sugere estratégias de correção,

o que contribui para a interpretação dos resultados e para a resolução mais eficiente de problemas de integração.

4.3. Arquitetura do Sistema

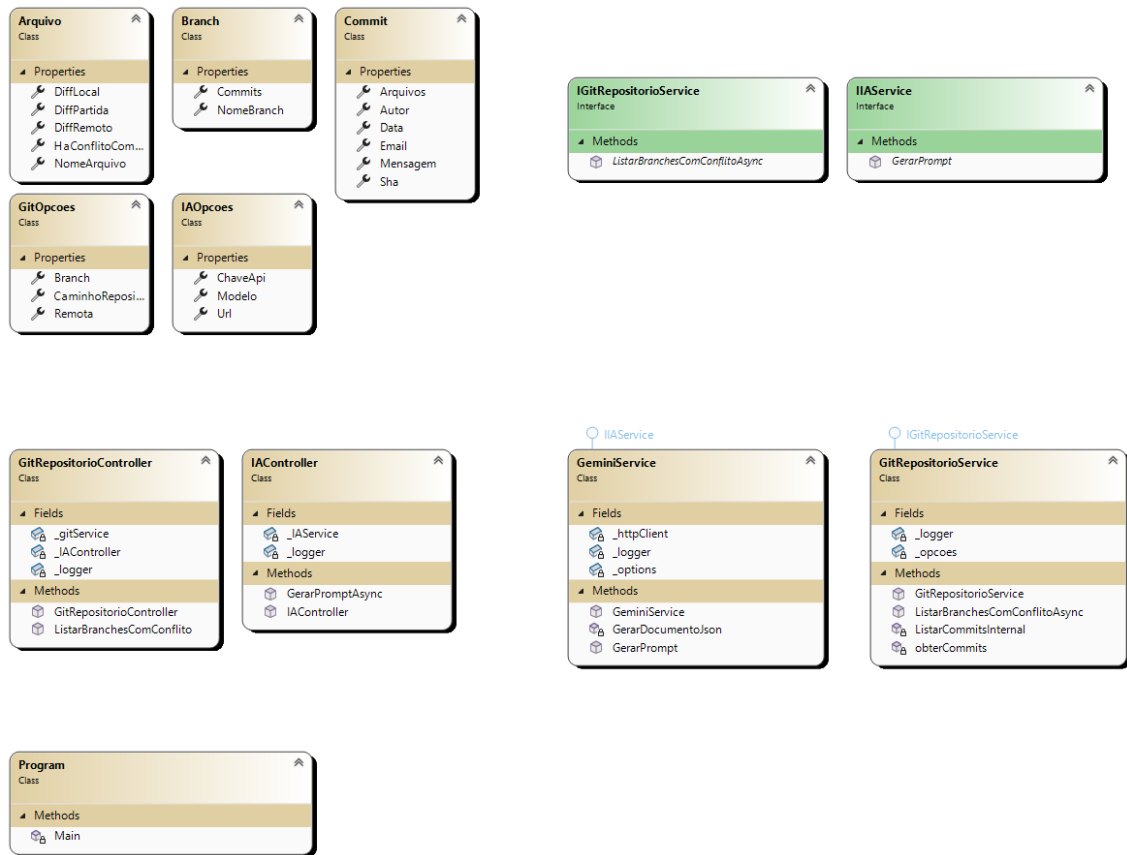


Figura 2. Arquitetura do Sistema

A arquitetura do sistema foi estruturada de forma modular, seguindo princípios de Arquitetura em Camadas combinados com conceitos de Arquitetura Hexagonal, com o objetivo de garantir organização, baixo acoplamento e facilidade de manutenção. O projeto está dividido em quatro níveis principais: Modelos, Serviços, Controladores e Camada de Entrada, cada qual responsável por uma função específica dentro do fluxo de execução (SENSEDIA, 2025; GUIA DEV, 2025).

4.3.1. Camada de Modelos

A camada de modelos concentra todas as entidades que representam os dados manipulados pelo sistema, como *Arquivo*, *Branch*, *Commit*, *GitOpcoes* e *IAOpcoes*. Essas classes possuem apenas propriedades, sem lógica de negócio, sendo responsáveis por estruturar e transportar informações entre as demais camadas. Essa separação permite que as regras de negócio permaneçam independentes das estruturas de dados.

As classes Arquivo, Branch e Commit entidades fundamentais que o notificador recebe ao consultar o repositório Git. Já o GitOpcoes e IAOpcoes são classes que descrevem os parâmetros de entrada do programa.

4.3.2. Camada de Serviços

Os serviços concentram a lógica principal do sistema, organizados por meio de interfaces e suas implementações:

- IGitRepositorioService/GitRepositorioService: Responsáveis por consultar o repositório Git, listar branches, identificar commits e detectar possíveis conflitos.
- IAService/GeminiService: Essa camada é responsável por implementar a comunicação com o modelo de inteligência artificial utilizado pelo sistema. Seu principal objetivo é analisar os arquivos identificados com potencial conflito e descrever possíveis estratégias de resolução para auxiliar o desenvolvedor. A implementação atual utiliza o serviço Gemini, que interpreta o contexto dos commits, compara diferenças e gera recomendações detalhadas. Entretanto, como essa comunicação é definida por meio da interface IAService, a arquitetura permite que outras LLMs sejam integradas futuramente sem alterar o restante da aplicação, garantindo flexibilidade e evolução tecnológica contínua.

4.3.3. Camada de Controladores

Os controladores atuam como mediadores entre a entrada do programa e os serviços:

- GitRepositorioController
- IAController

Eles coordenam o fluxo de execução, preparam as entradas, acionam os serviços necessários e retornam os resultados de maneira organizada. Essa camada garante que a lógica de alto nível não fique misturada com o código de infraestrutura.

4.3.4. Camada de Entrada (Program)

Representada pela classe Program, esta camada é o ponto inicial da aplicação. Ela configura dependências, controladores, serviços e dispara o fluxo principal.

5. Resultado

Como resultado da pesquisa, foi desenvolvido o componente NotificadorGit, responsável por monitorar o repositório remoto e identificar, de forma proativa, possíveis conflitos de código. O componente realiza a listagem das branches disponíveis, compara automaticamente suas alterações

com a branch local em uso e detecta arquivos potencialmente conflitantes. Para enriquecer o processo de análise, foi integrada uma LLM ao sistema — no estudo de caso, utilizou-se o modelo Gemini — permitindo a descrição contextualizada do conflito e a sugestão de possíveis estratégias de resolução. Esse resultado demonstra que a solução proposta é capaz de antecipar problemas de integração e fornecer suporte inteligente ao desenvolvedor durante o fluxo de versionamento.

Na sequência, são expostas as evidências da implementação, o caso de teste conduzidos e a demonstração do funcionamento do sistema desenvolvido.

Código Fonte: <https://github.com/devSantos16/NotificadorGit>

5.1. Caso de Teste

Para validar o funcionamento do *NotificadorGit*, foi definido um repositório de teste chamado *ConflitosSimulados* contendo duas branches: *main* e *feature/python*. Ambas possuem um arquivo denominado *arquivo_teste*, porém com conteúdo divergente na mesma linha específica:

- Branch **<origin/main>**: *"Este é um arquivo de teste em Markdown gerado automaticamente."*
- Branch **<origin/feature/python>**: *"Este é um arquivo de teste em Python gerado automaticamente."*

No ambiente local, o mesmo arquivo foi alterado para a seguinte mensagem:

- **Ambiente local**: *"Este é um arquivo de teste em Csharp gerado automaticamente."*

Esse cenário foi criado propositalmente para gerar um conflito direto entre a modificação local e a modificação existente na branch *python*. O objetivo do sistema é:

1. Identificar automaticamente que a branch *python* apresenta um potencial conflito com a alteração local.
2. Encapsular essas informações, descrevendo o trecho conflitante.
3. Gerar justificativas e orientações de resolução, utilizando o serviço de LLM (Gemini) para explicar o motivo do conflito e sugerir abordagens de correção.

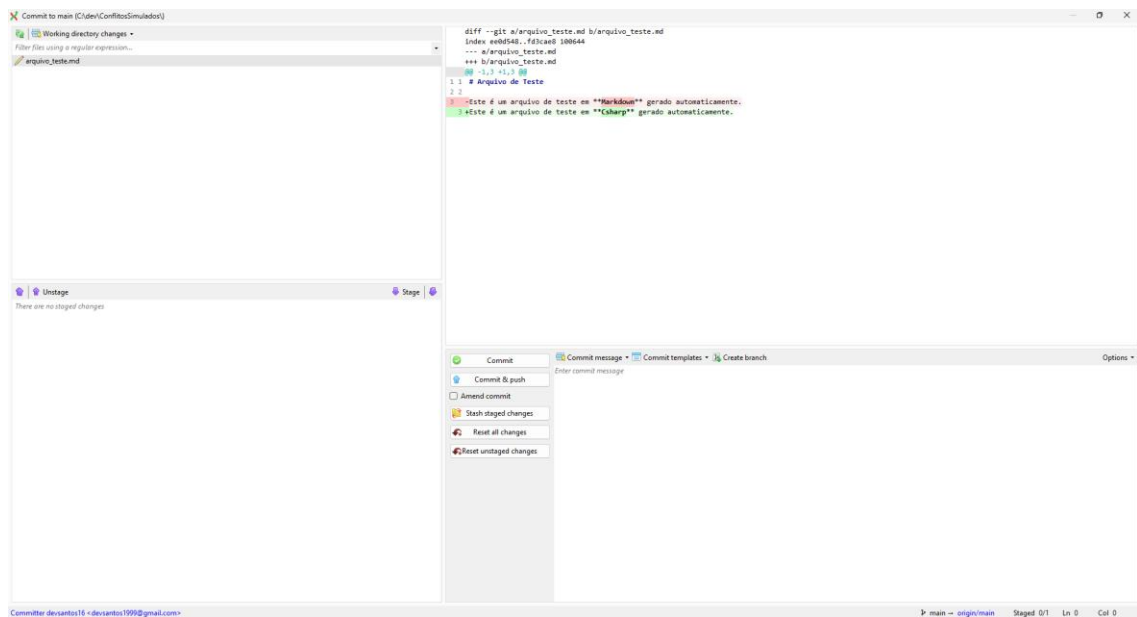


Figura 3. Diff do arquivo local com a branch <origin/main>

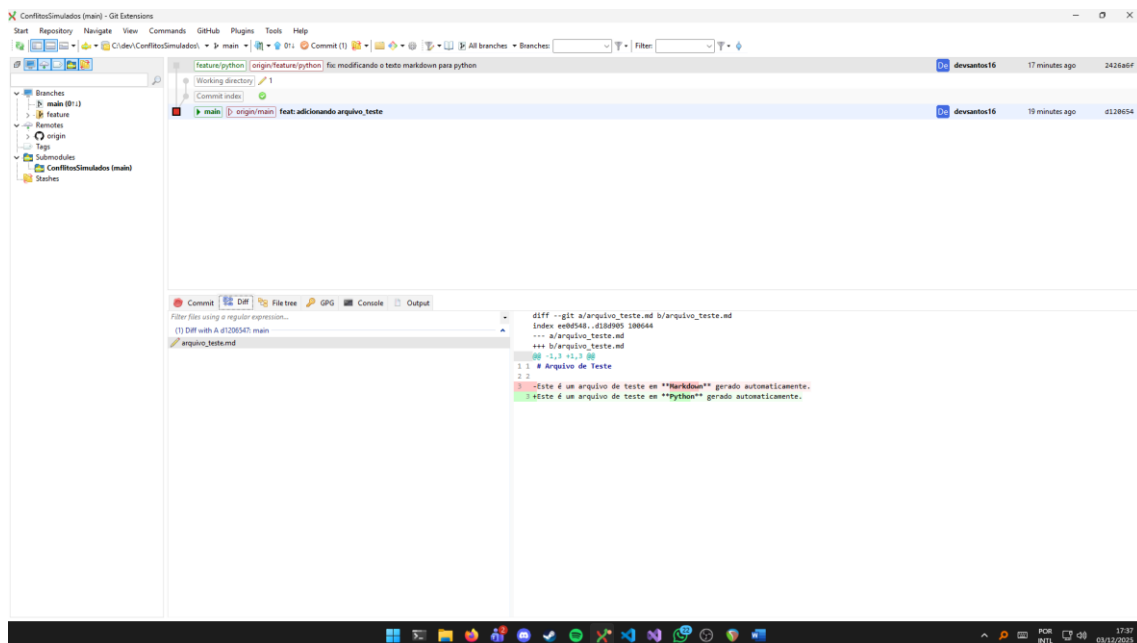


Figura 4. Panorama do repositório com evidências do arquivo local modificado pela Branch <origin/feature/python>

O repositório de teste se encontra em: <https://github.com/devSantos16/ConflitosSimulados>

5.2. Evidência de Implementação:

A partir do caso de teste elaborado, foi possível reunir evidências práticas da implementação, evidenciando o comportamento do protótipo frente às operações de listagem de branches,

comparação de alterações e detecção de conflitos. Esses resultados confirmam que as funcionalidades essenciais foram corretamente implementadas.

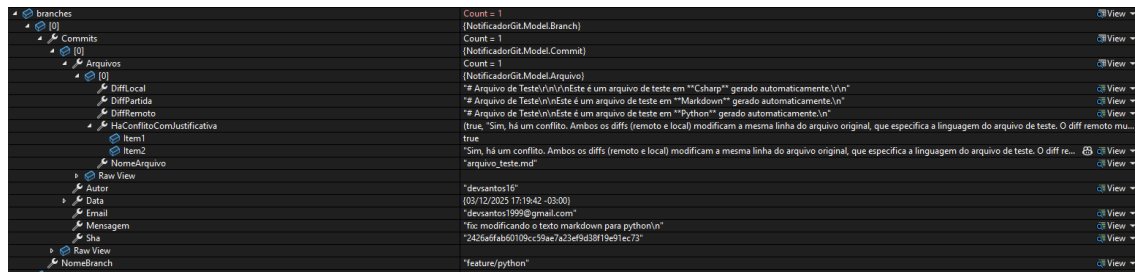


Figura 5. Implementação encapsulada em uma lista de Branches

A figura acima apresenta o resultado final obtido. O núcleo do notificador encapsula uma lista de *branches*, cada uma contendo seus respectivos *commits*, bem como os arquivos modificados e os *diffs* gerados. Esse conjunto de informações representa a estrutura consolidada que o sistema utiliza para identificar possíveis conflitos e monitorar alterações remotas.



Figura 6. Resposta do Serviço LLM encapsulada na propriedade HaConflitoComJustificativa

A Figura acima representa a resposta fornecida pelo serviço de LLM — neste projeto, a API do Gemini. Nessa resposta, o modelo identifica a existência de um conflito, descreve quais linhas foram afetadas (neste caso, a linha 3 do arquivo *arquivo_teste*) e orienta o usuário apresentando exemplos de possíveis soluções para resolver o conflito.

```
dbug: Inicializando obterCommits para análise de commits remotos.
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Finalizado obterCommits. Commits analisados=0 TempoMs=1
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Processando branch remota origin/feature/python
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Commits remotos a analisar para origin/feature/python: 1
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Inicializando obterCommits para análise de commits remotos.
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Analisando commit 2426a6fab60109cc59ae7a23ef9d38f19e91ec73 (Autor=devsantos16)
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Arquivo adicionado para análise: arquivo_teste.md (Commit=2426a6fab60109cc59ae7a23ef9d38f19e91ec73)
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Commit com conflitos detectado: 2426a6fab60109cc59ae7a23ef9d38f19e91ec73 (arquivos=1)
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Finalizado obterCommits. Commits analisados=1 TempoMs=24
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Commits conflitados encontrados em origin/feature/python: 1
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Finalizado ListarBranchesComConflito. Duraçao=1471ms. BranchesProcessadas=1
dbug: NotificadorGit.Service.GitRepositorioService[0]
dbug: Encontradas 1 branches com possível conflito.
dbug: NotificadorGit.Controller.GitRepositorioController[0]
dbug: Processando branch feature/python com 1 commits.
dbug: NotificadorGit.Controller.GitRepositorioController[0]
dbug: Processando commit 2426a6fab60109cc59ae7a23ef9d38f19e91ec73 (Autor: devsantos16) na branch feature/python.
dbug: NotificadorGit.Controller.GitRepositorioController[0]
dbug: Enviando prompt para IA. Branch=feature/python Commit=2426a6fab60109cc59ae7a23ef9d38f19e91ec73 ArquivoSnippet= Arquivo de Teste

Este é um arquivo de teste em **Python** gerado automaticamente...
dbug: NotificadorGit.Controller.IAController[0]
dbug: Enviando prompt para a IA.
dbug: System.Net.Http.HttpClient.IIAService.LogicalHandler[100]
dbug: Start processing HTTP request POST https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent
dbug: System.Net.Http.HttpClient.IIAService.ClientHandler[100]
dbug: Sending HTTP request POST https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent
dbug: System.Net.Http.HttpClient.IIAService.ClientHandler[101]
dbug: Received HTTP response headers after 4886.612ms - 200
dbug: System.Net.Http.HttpClient.IIAService.LogicalHandler[101]
dbug: End processing HTTP request after 4100.1286ms - 200
dbug: NotificadorGit.Controller.IAController[0]
dbug: Resposta recebida da IA.
dbug: NotificadorGit.Controller.GitRepositorioController[0]
dbug: Resposta IA obtida. Branch=feature/python Commit=2426a6fab60109cc59ae7a23ef9d38f19e91ec73 isTrue=True
dbug: NotificadorGit.Controller.GitRepositorioController[0]
dbug: Resposta IA (truncada): Sim, há um conflito. Ambos os diffs (remoto e local) modificam a mesma linha do arquivo original, que especifica a linguagem do arquivo de teste. O diff remoto muda "Markdown" para "Python", enquanto ...
dbug: NotificadorGit.Controller.GitRepositorioController[0]
```

Figura 7. Logs do processo

A figura acima ilustra o registro detalhado de logs gerados pelas camadas de *Controller* e *Service*, permitindo visualizar de forma clara o fluxo de execução da aplicação. Esses logs evidenciam as operações realizadas, auxiliam na identificação de comportamentos internos do sistema e contribuem para a validação do correto funcionamento dos processos implementados.

6. Discussão

Com base nas evidências apresentadas, observa-se que o *NotificadorGit* atendeu ao seu objetivo principal ao identificar potenciais conflitos entre alterações locais e remotas, além de monitorar o repositório por meio de logs estruturados e informações consolidadas. O sistema também demonstrou capacidade de fornecer suporte interpretativo por meio de uma LLM, enriquecendo a análise de conflitos com explicações detalhadas e sugestões de resolução. Esses resultados confirmam a eficácia do protótipo tanto na detecção antecipada de conflitos quanto no acompanhamento do fluxo de versionamento, validando a proposta e a implementação do módulo. Contudo, algumas limitações devem ser consideradas. O serviço *GeminiService* não garante precisão total na identificação de conflitos, apresentando falsos positivos em determinados testes — uma limitação inerente ao uso de modelos de linguagem que dependem do contexto do prompt é a

possibilidade de gerar interpretações incorretas, respostas imprecisas ou até “alucinações”, (UNIVERSITY OF MARY WASHINGTON – Library, 2024).

7. Conclusão

O presente trabalho buscou responder à questão de como um sistema de notificação inteligente pode auxiliar na detecção antecipada de conflitos em branches de código, reduzindo o retrabalho e aprimorando a comunicação entre desenvolvedores. Nesse contexto, o objetivo geral — desenvolver um sistema capaz de monitorar commits remotos e identificar alterações em arquivos modificados localmente — foi integralmente validado.

Essa validação foi possibilitada pela metodologia adotada, que estruturou de forma sistemática todas as etapas de desenvolvimento e experimentação, bem como pelo arcabouço teórico apresentado nas referências consultadas, as quais forneceram base conceitual e técnica para sustentar a construção e avaliação do projeto.

Como resultado, foi desenvolvido o componente *NotificadorGit*, que demonstrou ser capaz de monitorar proativamente o repositório remoto, listar as branches disponíveis e detectar arquivos potencialmente conflitantes por meio da comparação automática de alterações com a branch local.

As evidências apresentadas, baseadas em um caso de teste simulado, confirmaram a eficácia do protótipo tanto na detecção antecipada de conflitos quanto no acompanhamento do fluxo de versionamento. Um aspecto relevante foi a integração de uma *Large Language Model* (LLM) — o modelo Gemini — que enriqueceu a análise ao fornecer suporte interpretativo, com explicações contextualizadas do conflito e sugestões de estratégias de resolução.

Entretanto, algumas limitações devem ser consideradas. A precisão do serviço *GeminiService* não é totalmente garantida na identificação de conflitos, apresentando a possibilidade de falsos positivos — uma característica inerente a modelos de linguagem dependentes do contexto do prompt.

Para trabalhos futuros, destaca-se, como prioridade, a implementação de um sistema consumidor que amplie a usabilidade da solução. Recomenda-se, igualmente, a realização de testes de caixa branca e de caixa preta, a fim de aprimorar a testabilidade da aplicação e verificar sua disponibilidade em diferentes cenários operacionais. Ademais, a investigação do impacto da solução na redução de retrabalho e na melhoria da comunicação entre equipes distribuídas configura-se como um campo promissor para estudos adicionais, especialmente mediante validações conduzidas em ambientes reais de produção.

8. Agradecimentos

Agradeço à professora e ministra do curso, Josiane Brietzke Porto, pela oportunidade de desenvolver um projeto aplicado e pelo apoio oferecido ao longo do processo. Agradeço também ao meu ex-colega de trabalho, Alexandre Ballester, pela contribuição na definição da ideia e inspiração para o tema deste projeto.

9. Referências

RIBEIRO, B. B.; COSTA, C.; SANTOS, R. P. dos. *Understanding and Analyzing Factors that Affect Merge Conflicts from the Perspective of Software Developers*. Journal of Software Engineering Research and Development, v. 10, p. 12:1–12:17, 2022. DOI: <https://doi.org/10.5753/jserd.2022.2576>.

CHACON, Scott; STRAUB, Ben. *Pro Git*. 2. ed. Berkeley: Apress, 2014. Disponível em: <https://git-scm.com/book/en/v2>. Acesso em: 5 dez. 2025.

ADEPOJU, O. A.; ADEPOJU, A. A.; ADEPOJU, O. O. *Proactive Conflict Detection in Collaborative Software Development*. International Research Journal of Engineering and Technology (IRE Journals), v. 6, n. 7, p. 123–130, 2022. Disponível em: <https://www.irejournals.com/formatedpaper/1703431.pdf>. Acesso em: 5 dez. 2025.

BRUN, Y.; HOLMES, R.; ERNST, M. D.; NOTKIN, D. *Proactive detection of collaboration conflicts*. In: ACM SIGSOFT SYMPOSIUM ON THE FOUNDATIONS OF SOFTWARE ENGINEERING, 19.; EUROPEAN SOFTWARE ENGINEERING CONFERENCE, 13., 2011, Szeged. *Proceedings....* New York: ACM, 2011. p. 168–178. DOI: <https://doi.org/10.1145/2025113.2025139>.

MICROSOFT. C# documentation. Disponível em: <https://learn.microsoft.com/dotnet/csharp/>. Acesso em: 3 dez. 2025.

STACK OVERFLOW. Stack Overflow Developer Survey 2024. Disponível em: <https://survey.stackoverflow.co/2024/>. Acesso em: 3 dez. 2025.

LIBGIT2SHARP. LibGit2Sharp Documentation. Disponível em: <https://github.com/libgit2/libgit2sharp>. Acesso em: 3 dez. 2025.

SENSEDIA. *Padrão de Arquitetura Hexagonal e Sistemas Escaláveis*. Sensedia Blog, 21 maio 2025. Disponível em: <https://www.sensedia.com.br/post/uso-do-padrao-de-arquitetura-hexagonal>. Acesso em: 4 dez. 2025.

GUIA DEV. *Camadas e Padrões de Arquitetura*. Guia Dev, 2025. Disponível em: <<https://guia.dev/pt/pillars/software-architecture/layers-and-architecture-patterns.html>>. Acesso em: 4 dez. 2025.

GOOGLE. Gemini API Documentation. 2024. Disponível em: <https://ai.google.dev/gemini-api/docs>. Acesso em: 05 dez. 2025.

UNIVERSITY OF MARY WASHINGTON – Library. *AI for Research: Limitations*. 2024. Disponível em: <https://library.umw.edu/AI-for-research/limitations>. Acesso em: 05 dez. 2025.